FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA – FIAP
CURSO SUPERIOR DE TECNOLOGIA EM DEFESA CIBERNÉTICA
TURMA 1TDCOR – 2025

GRUPO AZANHA

Ryan Azanha

CHALLENGE DE MENTORIA I
Relatório Técnico Consolidado

Teste Ético de Segurança em Ambiente de Homologação Web

Versão pública – dados de identificação do alvo anonimizados

Barueri – SP
2025

GRUPO AZANHA

Ryan Azanha

CHALLENGE DE MENTORIA I

Relatório Técnico Consolidado

Relatório técnico apresentado à disciplina de Mentoria I do Curso Superior de Tecnologia em Defesa Cibernética da Faculdade de Informática e Administração Paulista – FIAP, como requisito parcial para avaliação do Challenge da disciplina – Turma 1TDCOR/2025, contemplando, em documento único, as quatro fases do desafio.

Barueri – SP

2025

NOTA SOBRE ANONIMIZAÇÃO

Este documento é a versão pública de um relatório de teste de intrusão realizado em ambiente acadêmico controlado. Por princípio de divulgação responsável, todos os dados que permitiriam identificar o ambiente avaliado foram removidos ou substituídos pelo marcador [REDIGIDO].

Foram anonimizados: o nome da aplicação e da organização responsável, o domínio e a URL pública, o endereço IP e o hostname do servidor, os caminhos internos da aplicação, os nomes dos cookies de sessão e os valores reais de tokens capturados durante o teste. O domínio alvo.example.com é um marcador de posição e não corresponde ao alvo real.

Nenhuma informação técnica relevante à compreensão do trabalho foi suprimida. Versões de software, identificadores CVE, vetores CVSS, comandos executados, achados, classificação de risco e plano de mitigação permanecem integralmente reproduzidos, uma vez que descrevem tecnologias de conhecimento público e não expõem o ambiente avaliado.

A anonimização segue a prática adotada por relatórios de consultoria divulgados publicamente e observa o princípio de que a demonstração de competência técnica não deve criar risco para terceiros.

SUMÁRIO EXECUTIVO

Este relatório documenta um teste ético de segurança (penetration test) executado, na modalidade Blackbox, sobre o ambiente de homologação da aplicação avaliada, acessível pelo endereço https://alvo.example.com. O trabalho seguiu a metodologia do NIST SP 800-115, utilizou o OWASP Top 10 (2021) como referência de classificação de falhas em aplicações web e aplicou o CVSS v3.1 para pontuação formal de risco.

Foram identificadas, validadas manualmente e priorizadas 16 vulnerabilidades. O achado de maior severidade é a execução de uma versão desatualizada do Apache Tomcat (9.0.93), acumulando 16 CVEs conhecidas, entre elas a CVE-2025-24813 (CVSS 9.8), que combina exposição de arquivos por equivalência de caminho e potencial de execução remota de código.

Quadro-resumo dos achados

Severidade	Qtd.	Principais achados
Crítica	1	CVE-2025-24813 – Apache Tomcat 9.0.93 (RCE / Path Equivalence)
Alta	4	CVEs múltiplas do Tomcat, CSRF token estático, ViewState JSF exposto, JWT validado no client-side
Média	10	Headers HTTP de segurança ausentes, cookies sem HttpOnly/Secure, PrimeFaces 6.1, postMessage com origem curinga, recursos externos sem SRI
Baixa/Info	1 + 1	URL de ambiente exposta no HTML; comentários de código revelando arquitetura

Fonte: Elaborado pelo autor (2025), com base na classificação CVSS v3.1 consolidada.

Além do diagnóstico, o relatório entrega um plano de mitigação dividido em três horizontes — hotfix, workaround e correção definitiva —, um roadmap com prazos e responsáveis sugeridos e critérios objetivos de aceitação para o reteste de cada vulnerabilidade. A atualização do Apache Tomcat para a versão 9.0.99 ou superior, isoladamente, resolve a maior parte dos achados críticos e altos.

Todas as provas de conceito foram construídas de forma não destrutiva: nenhuma alteração de dados, indisponibilidade ou impacto operacional foi provocada no ambiente avaliado. O trabalho respeitou integralmente o escopo autorizado e a legislação brasileira aplicável (Lei nº 12.737/2012, Lei nº 12.965/2014 e LGPD).

SUMÁRIO

NOTA SOBRE ANONIMIZAÇÃO
SUMÁRIO EXECUTIVO
1 INTRODUÇÃO
2 OBJETIVOS
2.1 Objetivo Geral
2.2 Objetivos Específicos
3 JUSTIFICATIVA E IMPORTÂNCIA DO TESTE ÉTICO DE SEGURANÇA
4 ESCOPO E LIMITES DE ATUAÇÃO
4.1 Ambiente Técnico Alvo
4.2 Atividades Permitidas e Proibidas
5 CRONOGRAMA E FRENTES DE TRABALHO
5.1 Cronograma de Execução
5.2 Frentes de Trabalho Executadas
6 FERRAMENTAS E FRAMEWORKS UTILIZADOS
7 METODOLOGIA DE ANÁLISE
8 CONSIDERAÇÕES ÉTICAS E LEGAIS
9 MAPEAMENTO DO AMBIENTE (FASE 2)
9.1 Reconhecimento Passivo com Dmitry
9.2 Enumeração de DNS com dnsrecon
9.3 Enumeração de Subdomínios com Sublist3r
9.4 Coleta OSINT com theHarvester
9.5 Análise de Rede com Scapy
9.6 Detecção de WAF com wafw00f
9.7 Verificação SNMP
9.8 Superfície de Ataque Consolidada
10 VARREDURA E IDENTIFICAÇÃO DE VULNERABILIDADES (FASE 3)
10.1 Scan de Vulnerabilidades com Nessus
10.2 Scan de Aplicação Web com Nikto
10.3 Análise Manual do Código-Fonte
10.4 Classificação CVSS v3.1 das Vulnerabilidades
11 VALIDAÇÃO E PROVAS DE CONCEITO (FASE 4)
11.1 Validação Manual e Eliminação de Falsos Positivos
11.2 Provas de Conceito (PoCs) Seguras
12 PLANO DE MITIGAÇÃO E ROADMAP (FASE 4)

12.1 Correções Imediatas (Hotfix)
12.2 Correções Temporárias (Workaround)
12.3 Correções Definitivas
12.4 Roadmap de Implementação
12.5 Plano de Retest e Critérios de Aceitação
13 RESULTADO ESPERADO E ALCANÇADO
14 CONCLUSÃO
GLOSSÁRIO
REFERÊNCIAS

LISTA DE TABELAS

Tabela 1 – Ambiente técnico alvo identificado
Tabela 2 – Escopo de atuação (permitido x proibido)
Tabela 3 – Cronograma de execução
Tabela 4 – Frentes de trabalho executadas
Tabela 5 – Ferramentas e frameworks utilizados
Tabela 6 – Superfície de ataque consolidada
Tabela 7 – Resumo dos achados do Nessus
Tabela 8 – Headers HTTP de segurança ausentes (Nikto)
Tabela 9 – Vulnerabilidades identificadas na análise manual
Tabela 10 – Classificação CVSS v3.1 consolidada
Tabela 11 – Plano de mitigação e roadmap
Tabela 12 – Critérios de aceitação para retest

1 INTRODUÇÃO

O presente relatório é resultado do Challenge de segurança ofensiva, atividade prática proposta pela disciplina de Mentoria I do curso Superior de Tecnologia em Defesa Cibernética da FIAP, turma 1TDCOR/2025. O desafio consiste na execução de um teste ético de segurança (ethical hacking) sobre o ambiente de homologação disponibilizado pela plataforma avaliada, acessível por meio do endereço https://alvo.example.com.

A crescente digitalização dos processos organizacionais ampliou significativamente a superfície de ataque das aplicações web, tornando indispensável a adoção de práticas estruturadas de avaliação de segurança. Nesse contexto, o pentest (penetration testing) consolida-se como uma abordagem metodológica capaz de identificar, classificar e priorizar vulnerabilidades antes que estas sejam exploradas por agentes maliciosos.

Este documento consolida, em volume único, as quatro fases previstas pelo Challenge: (i) Introdução e Proposta Detalhada; (ii) Mapeamento do Ambiente e Planejamento Operacional; (iii) Varredura, Identificação e Registro de Vulnerabilidades; e (iv) Validação, Mitigação e Relatório Final. A consolidação preserva a estrutura metodológica de cada fase, permitindo leitura sequencial do planejamento à entrega final.

2 OBJETIVOS

2.1 Objetivo Geral

Planejar, executar e documentar um teste ético de segurança no ambiente de homologação alvo.example.com, identificando, validando e classificando vulnerabilidades por meio de boas práticas de Ethical Hacking, e produzindo relatório técnico em conformidade com as normas da Associação Brasileira de Normas Técnicas (ABNT).

2.2 Objetivos Específicos

- Contextualizar a importância do teste ético de segurança no cenário atual de defesa cibernética;

- Definir o escopo, os limites e as restrições éticas de atuação durante o pentest;
- Elaborar um cronograma de execução com atividades, prazos e entregas;
- Selecionar as ferramentas e os frameworks utilizados em cada etapa do processo;
- Mapear o ambiente-alvo, identificando domínios, endereços IP, portas, serviços e tecnologias expostas;
- Executar varreduras controladas e identificar vulnerabilidades, classificando-as por meio do CVSS v3.1;
- Validar manualmente cada achado, eliminando falsos positivos e produzindo Provas de Conceito (PoCs) seguras;
- Propor plano de mitigação e roadmap de correção, com responsáveis e prazos;
- Documentar todas as decisões e premissas técnicas conforme padrão ABNT, garantindo rastreabilidade e auditabilidade.

3 JUSTIFICATIVA E IMPORTÂNCIA DO TESTE ÉTICO DE SEGURANÇA

O teste ético de segurança, também conhecido como pentest, constitui uma das principais práticas defensivas adotadas por organizações que buscam proteger seus ativos digitais. Diferentemente de um ataque malicioso, o pentest é executado mediante autorização formal, com escopo previamente definido e regras claras de conduta, com o propósito exclusivo de identificar fragilidades antes que sejam exploradas por terceiros.

No contexto deste challenge, a realização do teste em ambiente de homologação justifica-se por três motivos principais. Primeiro, sob o aspecto técnico, a prática permite aplicar, em cenário controlado e seguro, os conceitos de reconhecimento, varredura, análise de vulnerabilidades e elaboração de evidências, consolidando competências essenciais à atuação profissional em defesa cibernética.

Segundo, sob o aspecto pedagógico, o exercício promove a vivência integral do ciclo de um pentest — do planejamento à documentação final — exigindo organização, disciplina metodológica e capacidade de comunicação técnica, habilidades fundamentais para o profissional de segurança ofensiva e defensiva.

Terceiro, sob o aspecto ético e legal, o desafio reforça a importância da conduta responsável do profissional de cibersegurança, alinhando a prática às diretrizes do Código de Ética do Hacker Ético, à Lei nº 12.965/2014 (Marco Civil da Internet), à Lei nº 13.709/2018 (Lei Geral de Proteção de Dados Pessoais – LGPD) e às boas práticas internacionais reconhecidas pela OWASP, pelo NIST e pelo framework MITRE ATT&CK.

Dessa forma, a realização ordenada do teste sobre o ambiente alvo.example.com contribui para o amadurecimento técnico e ético do avaliador e para a entrega de um relatório que possa servir como subsídio à equipe responsável pelo ambiente, apoiando a melhoria contínua da postura de segurança da aplicação avaliada.

4 ESCOPO E LIMITES DE ATUAÇÃO

4.1 Ambiente Técnico Alvo

O alvo da avaliação corresponde à instância de homologação da aplicação avaliada, hospedada no endereço https://alvo.example.com. A Tabela 1 sintetiza as informações técnicas conhecidas sobre o ambiente, obtidas exclusivamente por meio de fontes públicas e de técnicas de reconhecimento passivo, sem qualquer interação intrusiva ou modificação de estado do alvo.

Tabela 1 – Ambiente técnico alvo identificado

Item	Informação identificada
Aplicação-alvo	Aplicação web corporativa (ambiente de homologação)
URL pública	https://alvo.example.com
Endereço IP resolvido	[REDIGIDO] – endereço IPv4 público único
Hostname associado	[REDIGIDO] – hostname de serviço de DNS dinâmico
Protocolo principal	HTTP/HTTPS (porta 80/tcp confirmada aberta em reconhecimento)
Subdomínios públicos	Nenhum identificado
Modalidade do teste	Blackbox (sem credenciais e sem conhecimento interno do ambiente)

Fonte: Elaborada pelo autor (2025), com base em reconhecimento passivo do alvo.

Cabe destacar que o teste foi conduzido na modalidade Blackbox, ou seja, sem qualquer conhecimento prévio da arquitetura interna, do código-fonte ou de

credenciais de acesso. Essa abordagem reproduz, de forma realista, o cenário enfrentado por um atacante externo que dispõe somente das informações publicamente expostas pelo alvo.

4.2 Atividades Permitidas e Proibidas

O escopo deste teste contemplou exclusivamente o ambiente de homologação avaliado, acessível pelo endereço https://alvo.example.com. Não foram realizadas atividades em quaisquer outros domínios, subdomínios ou serviços que não estivessem diretamente relacionados a este alvo, conforme delimitação expressa no enunciado do Challenge de Mentoria I.

A definição precisa do escopo é elemento crítico para a condução ética do pentest, uma vez que estabelece as fronteiras dentro das quais o avaliador está autorizado a atuar. A Tabela 2 sintetiza, de forma comparativa, as atividades incluídas no escopo de atuação (permitidas) e aquelas que estão fora do escopo (proibidas).

Tabela 2 – Escopo de atuação (permitido x proibido)

Dentro do escopo (Permitido)	Fora do escopo (Proibido)
Reconhecimento passivo (OSINT, DNS, WHOIS)	Ataques de negação de serviço (DoS/DDoS)
Varredura de portas e serviços expostos	Alteração ou exclusão de dados do alvo
Identificação de tecnologias e versões	Tentativa de força bruta em credenciais
Análise passiva e ativa controlada de aplicação web	Engenharia social contra usuários reais
Detecção de falhas comuns (OWASP Top 10)	Acesso a sistemas fora do domínio alvo.example.com
Análise de cabeçalhos HTTP e configurações	Exploração destrutiva ou que comprometa disponibilidade

Fonte: Elaborada pelo autor (2025).

Adicionalmente, foram respeitados os seguintes princípios: (i) não derrubar ou indisponibilizar o sistema; (ii) não alterar, criar ou excluir dados; (iii) não realizar tentativas de força bruta sobre credenciais; e (iv) interromper imediatamente qualquer atividade que apresentasse risco de impacto operacional ao ambiente.

5 CRONOGRAMA E FRENTES DE TRABALHO

5.1 Cronograma de Execução

O cronograma foi estruturado considerando o prazo de entrega final do Challenge e a divisão das atividades nas quatro fases previstas. A Tabela 3 apresenta a distribuição das atividades.

Tabela 3 – Cronograma de execução

Semana	Atividade	Status
1	Definição de escopo, regras de conduta e planejamento	Concluído
1	Estudo das normas ABNT e estruturação do relatório	Concluído
2	Levantamento bibliográfico (OWASP, NIST, MITRE ATT&CK)	Concluído
2	Reconhecimento e mapeamento do ambiente (Fase 2)	Concluído
3	Varredura e identificação de vulnerabilidades (Fase 3)	Concluído
3	Validação, PoCs e plano de mitigação (Fase 4)	Concluído
4	Revisão final, formatação ABNT e entrega	Concluído

Fonte: Elaborada pelo autor (2025).

5.2 Frentes de Trabalho Executadas

A execução foi organizada em frentes técnicas complementares, de modo a garantir cobertura sobre todas as etapas críticas do pentest, do reconhecimento inicial à proposição do plano de correção. A Tabela 4 descreve cada frente e suas atribuições.

Tabela 4 – Frentes de trabalho executadas

Frente	Atribuições principais
Reconhecimento	OSINT, fingerprinting, enumeração de DNS e mapeamento de superfície de ataque
Varredura	Varredura de portas e serviços, scan de vulnerabilidades de infraestrutura com Nessus
Aplicação Web	Nikto, OWASP ZAP, Burp Suite e análise manual de código-fonte HTML/JavaScript
Validação e PoC	Eliminação de falsos positivos, construção de provas de conceito não destrutivas
Classificação e Mitigação	Pontuação CVSS v3.1, priorização de risco, roadmap e critérios de retest
Documentação	Redação técnica em padrão ABNT, consolidação de evidências e rastreabilidade

Fonte: Elaborada pelo autor (2025).

6 FERRAMENTAS E FRAMEWORKS UTILIZADOS

A seleção das ferramentas e frameworks foi orientada por dois critérios principais: (i) ampla aceitação na comunidade de segurança ofensiva e (ii) compatibilidade com a metodologia adotada. Todas as ferramentas listadas são de código aberto ou possuem versão comunitária gratuita, e foram executadas a partir de uma estação de trabalho com sistema Kali Linux. A Tabela 5 resume as ferramentas utilizadas e suas finalidades.

Tabela 5 – Ferramentas e frameworks utilizados

Ferramenta	Categoria	Finalidade no projeto
Dmitry	Reconhecimento passivo	Coleta WHOIS, DNS, e-mails e portas
theHarvester	OSINT	Coleta de e-mails, hosts e subdomínios
dnsrecon	Enumeração DNS	Varredura completa de registros DNS
Sublist3r	Enumeração de subdomínios	Coleta via múltiplos motores de busca
Scapy	Análise de rede	Traceroute e manipulação de pacotes
wafw00f	Detecção de WAF	Identificação de Web Application Firewall
Nmap	Varredura de rede	Identificação de portas, serviços e SO
Nessus Essentials	Scanner de vulnerabilidades	Identificação de CVEs em infraestrutura
Nikto	Scanner web	Verificação de configurações inseguras
OWASP ZAP	Proxy de interceptação	Detecção passiva e ativa em aplicações web
Burp Suite	Proxy de interceptação	Análise de requisições HTTP/HTTPS
wget / curl	Coleta de conteúdo	Download e inspeção de respostas HTTP
Kali Linux	Sistema operacional	Plataforma para execução das ferramentas

Fonte: Elaborada pelo autor (2025).

Quanto aos frameworks de referência, foram adotados três pilares complementares. O OWASP (Open Worldwide Application Security Project) serviu de base para a classificação de vulnerabilidades em aplicações web, especialmente o OWASP Top 10 (2021) e o OWASP Web Security Testing Guide (WSTG). O NIST SP 800-115 (Technical Guide to Information Security Testing and Assessment) orientou a estruturação geral do processo de teste, e o framework MITRE ATT&CK serviu de referência para o mapeamento de táticas, técnicas e procedimentos (TTPs) potencialmente observáveis no ambiente.

7 METODOLOGIA DE ANÁLISE

A metodologia adotada seguiu uma abordagem top-down, estruturada em fases sequenciais e iterativas, com base nos princípios estabelecidos pelo NIST SP 800-115 e adaptada ao escopo definido para o Challenge de Mentoria I. As fases executadas foram:

- **Fase 1 – Planejamento:** definição de objetivos, escopo, regras de conduta, cronograma e entregas, materializada nas seções iniciais deste relatório;
- **Fase 2 – Reconhecimento e mapeamento:** levantamento da superfície de ataque, identificação de domínios, endereços IP, endpoints, serviços expostos e tecnologias utilizadas, com prevalência de técnicas passivas (OSINT) e ativas controladas;
- **Fase 3 – Varredura e identificação de vulnerabilidades:** execução de scans de portas e serviços, análise passiva e ativa de aplicação web, validação manual de falsos positivos e classificação de risco por meio do CVSS v3.1;
- **Fase 4 – Validação, mitigação e relatório final:** elaboração de Provas de Conceito (PoC) seguras, proposição de medidas corretivas (hotfix, workaround e correção definitiva), construção de roadmap e definição do plano de retest.

A análise de risco utilizou a métrica CVSS v3.1 (Common Vulnerability Scoring System), considerando os vetores de exploração, complexidade do ataque, requisitos de privilégio, interação do usuário, escopo e impacto sobre confidencialidade, integridade e disponibilidade. Os achados foram classificados nos níveis Crítico, Alto, Médio, Baixo e Informativo, e cada vulnerabilidade verificada foi registrada com evidências reproduzíveis (comandos, parâmetros e timestamps), respeitando-se rigorosamente o escopo definido.

8 CONSIDERAÇÕES ÉTICAS E LEGAIS

A condução do teste observou integralmente os princípios éticos da segurança ofensiva, sintetizados nos seguintes compromissos: (i) atuar exclusivamente dentro do escopo autorizado; (ii) preservar a integridade, a confidencialidade e a disponibilidade dos dados do ambiente; (iii) registrar e relatar todas as atividades executadas; e (iv) comunicar imediatamente quaisquer achados sensíveis aos responsáveis pelo desafio.

Além disso, a atuação respeitou a legislação brasileira vigente, em especial a Lei nº 12.737/2012 (Lei Carolina Dieckmann), que tipifica os crimes informáticos; a Lei nº 12.965/2014 (Marco Civil da Internet), que estabelece os princípios para o uso da Internet no Brasil; e a Lei nº 13.709/2018 (LGPD), que regula o tratamento de dados pessoais. Nenhuma informação pessoal foi coletada, armazenada ou compartilhada fora do contexto acadêmico deste Challenge.

9 MAPEAMENTO DO AMBIENTE (FASE 2)

Esta seção apresenta os resultados da Fase 2 do Challenge, que teve como objetivo realizar o levantamento detalhado do ambiente-alvo, identificando domínios, endereços IP, endpoints e serviços expostos, com vistas a construir uma visão completa da superfície de ataque. Todas as atividades foram conduzidas com técnicas predominantemente passivas e ativas controladas, sem qualquer impacto sobre a disponibilidade do ambiente.

As ferramentas utilizadas nesta fase foram executadas a partir de uma estação Kali Linux atualizada, e os comandos exatos, parâmetros e saídas resumidas são apresentados nas subseções a seguir, garantindo a reprodutibilidade do trabalho.

9.1 Reconhecimento Passivo com Dmitry

A ferramenta Dmitry (Deepmagic Information Gathering Tool) foi utilizada como ponto de partida do reconhecimento, com o objetivo de coletar o máximo de informações públicas sobre o host. A ferramenta consulta WHOIS, registros DNS, e-mails associados ao domínio e realiza uma varredura inicial de portas TCP. O comando executado foi:

```
dmitry -wmnspb alvo.example.com -o dmitry-resultado
```

Os parâmetros indicam: -w (WHOIS de domínio), -m (WHOIS de e-mails), -n (lookup netcraft.com), -s (busca por subdomínios), -p (port scan TCP) e -b (banner grabbing). A opção -o salva o resultado em arquivo. As principais informações obtidas foram:

- Endereço IP resolvido: um endereço IPv4 público único [REDIGIDO];
- Porta 80/tcp aberta (HTTP);
- Nenhum subdomínio público identificado pelo Dmitry;

- Nenhum e-mail público associado ao domínio.

9.2 Enumeração de DNS com dnsrecon

Para complementar o levantamento DNS, foi utilizada a ferramenta dnsrecon, script Python nativo do Kali Linux capaz de realizar varreduras completas de DNS, identificar registros (A, AAAA, MX, NS, SOA, TXT) e detectar provedores de hospedagem em nuvem associados ao domínio. O comando executado foi:

```
dnsrecon -d alvo.example.com -t std -j resultado.json
cat resultado.json
```

O parâmetro -t std executa o conjunto padrão de testes (consulta SOA, NS, A, AAAA, MX, SRV, TXT) e -j exporta o resultado em formato JSON estruturado. O resultado confirmou o endereço IP principal e revelou registros DNS adicionais, sendo este um dos pontos centrais do mapeamento da infraestrutura.

9.3 Enumeração de Subdomínios com Sublist3r

A ferramenta Sublist3r foi utilizada com o objetivo específico de identificar subdomínios associados a alvo.example.com, utilizando múltiplos motores de busca (Google, Bing, Yahoo, Baidu, Ask, Netcraft, VirusTotal e ThreatCrowd) de forma simultânea. O comando executado foi:

```
sublist3r -d alvo.example.com
```

Nenhum subdomínio público foi descoberto, o que sugere que o domínio alvo.example.com possui exposição externa restrita ao host principal, reduzindo a superfície de ataque acessível a um atacante externo.

9.4 Coleta OSINT com theHarvester

Foi utilizado o theHarvester para tentar coletar informações públicas adicionais (e-mails, hosts, nomes de pessoas e perfis em redes sociais) associados ao domínio. A ferramenta integra-se a diversos motores de busca e fontes públicas como LinkedIn, Bing, Google e Shodan.

Apesar da varredura, nenhuma informação relevante foi retornada para o domínio em questão, reforçando o cenário de baixa exposição pública observado nas ferramentas anteriores.

9.5 Análise de Rede com Scapy

A biblioteca Scapy foi empregada para realizar um traceroute customizado em direção ao alvo, permitindo o mapeamento da rota percorrida pelos pacotes entre a estação de origem e o servidor de destino. Esse mapeamento auxilia na identificação de saltos intermediários, possíveis equipamentos de borda e operadoras envolvidas na rota. O código executado em ambiente Python foi:

```
from scapy.all import traceroute

ans, unans = traceroute(["alvo.example.com"], maxttl=20, dport=80)
```

Os parâmetros maxttl=20 e dport=80 limitam o número máximo de saltos a 20 e direcionam o teste à porta HTTP.

9.6 Detecção de WAF com wafw00f

Para verificar a presença de mecanismos de proteção de borda (Web Application Firewall – WAF), foi utilizada a ferramenta wafw00f, capaz de detectar mais de 150 produtos WAF distintos por meio de assinaturas específicas em respostas HTTP. O comando executado foi:

```
wafw00f alvo.example.com
```

A varredura indicou que nenhum WAF foi detectado na frente da aplicação, o que representa um ponto crítico para a postura defensiva do ambiente, uma vez que a ausência de WAF facilita a exploração de vulnerabilidades comuns como injeção SQL, XSS e tentativas de evasão de filtros.

9.7 Verificação SNMP

Foi realizada uma verificação do protocolo SNMP (Simple Network Management Protocol), amplamente utilizado em sistemas de monitoramento de rede. Quando configurado de forma inadequada (com communities padrão como public ou private), o SNMP pode expor informações sensíveis sobre interfaces, processos e configurações do servidor.

A verificação demonstrou que o SNMP encontra-se desabilitado ou bloqueado externamente no servidor avaliado, o que constitui um ponto positivo para a postura de segurança.

9.8 Superfície de Ataque Consolidada

A partir das atividades de reconhecimento realizadas nesta fase, foi possível construir uma visão consolidada da superfície de ataque do ambiente alvo.example.com, apresentada na Tabela 6.

Tabela 6 – Superfície de ataque consolidada

Categoria	Resultado
IP público	[REDIGIDO] – endereço IPv4 público único
Hostname associado	[REDIGIDO] – hostname de serviço de DNS dinâmico
Portas TCP abertas	80/tcp (HTTP)
Subdomínios públicos	Nenhum identificado
WAF (Web Application Firewall)	Não detectado
SNMP	Desabilitado ou bloqueado externamente
E-mails públicos	Nenhum identificado
Servidor web identificado	Apache Tomcat 9.0.93 (confirmado posteriormente)
Provedor de DNS dinâmico	DynDNS (via hostname [REDIGIDO])

Fonte: Elaborada pelo autor (2025), com base nos resultados das ferramentas Dmitry, dnsrecon, Sublist3r, theHarvester, wafw00f e Scapy.

A análise da superfície de ataque revelou um ambiente com exposição externa restrita ao host principal, mas com ausência de mecanismos defensivos importantes (WAF), o que justifica o aprofundamento da análise por meio de varreduras de vulnerabilidade especializadas, apresentadas na próxima seção.

10 VARREDURA E IDENTIFICAÇÃO DE VULNERABILIDADES (FASE 3)

Esta seção apresenta os resultados da Fase 3 do Challenge, dedicada à execução de varreduras controladas e análises ativas sobre o ambiente de homologação, com o objetivo de identificar vulnerabilidades, validar sua existência por meio de verificação manual, classificá-las quanto à severidade e gerar evidências reproduzíveis. Toda a execução respeitou os limites estabelecidos no escopo (seção 4), evitando ações destrutivas ou de impacto operacional.

10.1 Scan de Vulnerabilidades com Nessus

A ferramenta Nessus Essentials foi utilizada para realizar um scan completo de vulnerabilidades de infraestrutura sobre o host público do ambiente-alvo. O Nessus é

um scanner amplamente reconhecido no mercado, mantido pela Tenable, que correlaciona versões de software identificadas com bases de CVE públicas, gerando achados classificados por severidade.

O scan foi configurado no modo Basic Network Scan, sem credenciais, respeitando o escopo Blackbox definido. A Tabela 7 sintetiza os achados retornados.

Tabela 7 – Resumo dos achados do Nessus

Severidade	Quantidade	Observação
Críticas	5	CVEs acumuladas no Apache Tomcat 9.0.93
Altas	9	DoS, bypass de autenticação, HTTP request smuggling
Médias	1	Arquivos padrão expostos publicamente
Informativas	31	Banner grabbing, detecção de versão, entre outros

Fonte: Elaborada pelo autor (2025), com base no relatório do Nessus.

10.1.1 Apache Tomcat 9.0.93 – Vulnerabilidade Principal

O achado mais crítico identificado pelo Nessus refere-se à versão do Apache Tomcat em execução no servidor: a versão 9.0.93, vulnerável a 16 CVEs documentadas. A versão segura corresponde à 9.0.99 ou superior. Dentre as CVEs identificadas, destaca-se a CVE-2025-24813, que apresenta dois vetores de exploração:

- **Information Disclosure / Path Equivalence:** permite que um atacante acesse arquivos sensíveis manipulando o nome do arquivo com um ponto interno (ex.: file.Name), enganando o Tomcat e expondo conteúdo protegido;
- **Remote Code Execution (RCE):** caso o servidor possua escrita habilitada no Default Servlet, suporte a partial PUT ativo, sessões persistentes em arquivo e bibliotecas que permitam desserialização, é possível a execução remota de código arbitrário.

Adicionalmente à CVE-2025-24813, foram identificadas vulnerabilidades de HTTP request smuggling, bypass de autenticação, denial of service e information disclosure, todas relacionadas à versão desatualizada do Apache Tomcat. Todas essas vulnerabilidades são integralmente mitigadas com a atualização da versão para 9.0.99 ou superior.

10.2 Scan de Aplicação Web com Nikto

Para a análise específica da camada de aplicação web, foi utilizado o Nikto, scanner de vulnerabilidades amplamente adotado que identifica configurações inseguras, headers HTTP ausentes ou mal configurados, arquivos padrão expostos e outras falhas conhecidas em servidores web. O comando executado foi:

```
nikto -h alvo.example.com -o nikto-resultado.json -Format json
```

Foram identificados seis cabeçalhos HTTP de segurança ausentes no servidor, conforme detalhado na Tabela 8.

Tabela 8 – Headers HTTP de segurança ausentes (Nikto)

Header ausente	Risco	Função do header
Strict-Transport-Security	Médio	Força o uso de HTTPS, evitando downgrade para HTTP
Content-Security-Policy	Alto	Previne XSS e injeção de conteúdo malicioso
X-Content-Type-Options	Médio	Impede MIME-sniffing pelo navegador
Referrer-Policy	Baixo	Controla informações de referência entre páginas
Permissions-Policy	Baixo	Controla acesso a recursos do browser (câmera, microfone)
X-Frame-Options	Médio	Previne clickjacking (substituível por CSP frame-ancestors)

Fonte: Elaborada pelo autor (2025), com base na saída do Nikto.

A ausência conjunta destes cabeçalhos indica falta de hardening básico no servidor, configuração que deveria estar presente em qualquer aplicação web em produção ou homologação. Estes achados, individualmente de severidade média a baixa, podem ser combinados a outras vulnerabilidades para potencializar o impacto de ataques como XSS e clickjacking.

10.3 Análise Manual do Código-Fonte

Para complementar as varreduras automatizadas, foi realizada uma análise manual do código-fonte HTML e JavaScript da página principal da aplicação. O conteúdo foi obtido por meio do comando:

```
wget https://alvo.example.com/ -O index.html
```

Em seguida, o arquivo foi inspecionado em busca de padrões inseguros, tokens estáticos, validações realizadas no lado do cliente, comentários expostos e referências a versões de bibliotecas. Os achados são apresentados nas subseções a seguir.

10.3.1 CSRF Token Estático e Duplicado

Foi identificado que o token CSRF aparece duas vezes no HTML da página de login, com o mesmo valor estático, conforme exemplificado abaixo:

```
<input type="hidden" value="[TOKEN-REDIGIDO]"
       name="csrfToken" id="csrfToken" />
```

Um token CSRF que não é regenerado a cada requisição perde completamente sua função protetora contra ataques de Cross-Site Request Forgery, uma vez que pode ser previsto ou capturado uma única vez e reutilizado por um atacante.

10.3.2 ViewState do JSF Exposto

O parâmetro javax.faces.ViewState, característico de aplicações JSF (JavaServer Faces), foi identificado no HTML com valor aparentemente estático e repetido em dois formulários distintos:

```
<input type="hidden" name="javax.faces.ViewState"
       id="j_id1:javax.faces.ViewState:0"
       value="[VIEWSTATE-REDIGIDO]"
       autocomplete="off" />
```

Caso não exista proteção server-side adequada (criptografia ou assinatura HMAC do ViewState), este parâmetro pode ser explorado em ataques de ViewState Deserialization, vulnerabilidade conhecida em aplicações JSF/PrimeFaces, com potencial de execução remota de código.

10.3.3 postMessage com Origem Curinga

Foi identificado o uso de window.postMessage com o targetOrigin configurado como curinga, conforme exemplo:

```
window.parent.postMessage('{"evento": "TITULO_PAGINA", ...}', '*');
```

O uso do curinga como targetOrigin permite que qualquer página em qualquer domínio receba as mensagens postadas. Isso pode vazar informações sensíveis (URL atual, título da página, dados de sessão) para iframes ou janelas maliciosas que carreguem a aplicação.

10.3.4 Validação Client-Side de Cookie JWT

Identificou-se que a lógica de autenticação baseada em cookies é verificada no JavaScript do lado do cliente:

```
if (getCookie("SESSION") == "JWT_TOKEN") {
    // lógica de redirecionamento autenticado
}
```

Validações de autenticação realizadas exclusivamente no client-side podem ser facilmente manipuladas via DevTools do navegador. Um atacante pode forjar o cookie de sessão e alterar o fluxo de redirecionamento, embora o acesso efetivo aos recursos protegidos dependa também das validações server-side.

10.3.5 Outros Achados Relevantes

A análise manual identificou ainda os seguintes pontos:

- **URL de ambiente interno hardcoded:** o endereço https://alvo.example.com/appRedigidoWeb aparece explicitamente no HTML, revelando infraestrutura de desenvolvimento;
- **Cookies sem flags HttpOnly e Secure:** os cookies de sessão documentados no próprio HTML (cinco cookies próprios da aplicação) não apresentam menção a estas flags, tornando-os acessíveis via JavaScript (vetor para roubo via XSS) e potencialmente transmissíveis em conexões HTTP;
- **Versão antiga do PrimeFaces:** o caminho /appRedigido/javax.faces.resource/...jsf?ln=primefaces&v=6.1 expõe a versão exata da biblioteca PrimeFaces (6.1), considerada antiga e com CVEs conhecidas, especialmente envolvendo EL injection e FileUpload;
- **Recursos externos sem SRI:** o carregamento de recursos externos (ex.: Google Fonts) sem o atributo integrity (Subresource Integrity) cria dependência de integridade no servidor externo;
- **Comentários de código que revelam arquitetura:** comentários no HTML expõem integrações desativadas (CAS, Twitter, Facebook, Google, Azure), revelando a arquitetura do sistema.

A Tabela 9 consolida as vulnerabilidades identificadas na análise manual do código-fonte.

Tabela 9 – Vulnerabilidades identificadas na análise manual

Vulnerabilidade	Categoria OWASP	Severidade inicial
CSRF token estático	A01:2021 – Broken Access Control	**Crítico**
javax.faces.ViewState exposto	A05:2021 – Security Misconfiguration	**Alto**

Vulnerabilidade	Categoria OWASP	Severidade inicial
postMessage com origem curinga	A03:2021 – Injection	**Crítico**
Cookies sem HttpOnly/Secure	A05:2021 – Security Misconfiguration	**Alto**
JWT validado no client-side	A07:2021 – Auth Failures	**Alto**
URL hardcoded no HTML	A05:2021 – Security Misconfiguration	**Crítico**
PrimeFaces 6.1 (versão antiga)	A06:2021 – Vulnerable Components	**Médio**
Recursos externos sem SRI	A08:2021 – Software Integrity Failures	**Médio**
Comentários expondo arquitetura	A05:2021 – Security Misconfiguration	**Médio**

Fonte: Elaborada pelo autor (2025).

10.4 Classificação CVSS v3.1 das Vulnerabilidades

Após a consolidação dos achados das três frentes (Nessus, Nikto e análise manual), cada vulnerabilidade foi classificada formalmente segundo o framework Common Vulnerability Scoring System versão 3.1 (CVSS v3.1), mantido pelo FIRST (Forum of Incident Response and Security Teams).

O CVSS v3.1 gera uma pontuação numérica de 0,0 a 10,0 com base em um vetor composto pelas seguintes métricas base:

- **AV (Attack Vector):** vetor de ataque – Network (N), Adjacent (A), Local (L), Physical (P);
- **AC (Attack Complexity):** complexidade do ataque – Low (L), High (H);
- **PR (Privileges Required):** privilégios requeridos – None (N), Low (L), High (H);
- **UI (User Interaction):** interação do usuário – None (N), Required (R);
- **S (Scope):** escopo – Unchanged (U), Changed (C);
- **C/I/A:** impacto em Confidencialidade, Integridade e Disponibilidade – None (N), Low (L), High (H).

A Tabela 10 apresenta a classificação CVSS v3.1 consolidada de todas as vulnerabilidades identificadas.

Tabela 10 – Classificação CVSS v3.1 consolidada

ID	Vulnerabilidade	Score	Vetor CVSS v3.1	Severidade
AZANHA-01	CVE-2025-24813 (Apache Tomcat – RCE)	9.8	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H	**Crítica**

ID	Vulnerabilidade	Score	Vetor CVSS v3.1	Severidade
AZANHA-02	Apache Tomcat – CVEs múltiplas (Altas)	8.1	AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H	**Alta**
AZANHA-03	Apache Tomcat – Arquivos padrão expostos	5.3	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N	**Média**
AZANHA-04	CSRF token estático e reutilizável	8.8	AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H	**Alta**
AZANHA-05	ViewState JSF exposto (potencial RCE)	8.1	AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H	**Alta**
AZANHA-06	postMessage com targetOrigin curinga	6.5	AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:N/A:N	**Média**
AZANHA-07	JWT validado em client-side	7.5	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N	**Alta**
AZANHA-08	Cookies sem HttpOnly/Secure	6.5	AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:N/A:N	**Média**
AZANHA-09	PrimeFaces 6.1 desatualizado	6.1	AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N	**Média**
AZANHA-10	Content-Security-Policy ausente	5.4	AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N	**Média**
AZANHA-11	Strict-Transport-Security ausente	4.8	AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:L/A:N	**Média**
AZANHA-12	X-Content-Type-Options ausente	4.3	AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N	**Média**
AZANHA-13	X-Frame-Options ausente (Clickjacking)	4.3	AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N	**Média**
AZANHA-14	URL de ambiente exposta em HTML	3.1	AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N	**Baixa**
AZANHA-15	Recursos externos sem SRI	5.3	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N	**Média**
AZANHA-16	Comentários expondo arquitetura	0.0	Informativo	**Info**

Fonte: Elaborada pelo autor (2025), com base na especificação CVSS v3.1 (FIRST, 2019).

11 VALIDAÇÃO E PROVAS DE CONCEITO (FASE 4)

Esta seção apresenta os resultados da Fase 4 do Challenge, dedicada à validação manual das vulnerabilidades identificadas, à eliminação de falsos positivos e à elaboração de Provas de Conceito (PoCs) seguras, que demonstram a existência

das falhas sem causar qualquer impacto operacional ou modificação de dados no ambiente.

11.1 Validação Manual e Eliminação de Falsos Positivos

Todas as vulnerabilidades identificadas pelas ferramentas automatizadas (Nessus e Nikto) foram submetidas a verificação manual para confirmar sua aplicabilidade ao contexto do ambiente avaliado. O processo de validação seguiu três etapas:

- Reexecução do teste em condições controladas, registrando comando, horário (timestamp) e versão da ferramenta;
- Inspeção manual da resposta retornada pela aplicação, verificando se o comportamento observado confirma a presença da vulnerabilidade;
- Pesquisa em base CVE/NVD e em literatura técnica para validar a exploração potencial no contexto da versão e configuração identificadas.

Durante o processo, foram descartadas três detecções que se revelaram falsos positivos: dois alertas informativos do Nessus relativos a serviços não expostos externamente e um aviso do Nikto sobre cabeçalho que, na verificação manual via curl, mostrou-se presente em respostas específicas. As 16 vulnerabilidades remanescentes, apresentadas na Tabela 10, foram confirmadas e seguem documentadas neste relatório.

11.2 Provas de Conceito (PoCs) Seguras

Para as vulnerabilidades de maior criticidade, foram elaboradas Provas de Conceito seguras, com o objetivo único de demonstrar a existência da falha. Nenhuma das PoCs causou alteração de dados, indisponibilidade ou impacto operacional ao ambiente avaliado. As PoCs estão organizadas por ID da vulnerabilidade.

11.2.1 PoC AZANHA-01 – CVE-2025-24813 (Apache Tomcat)

Objetivo: confirmar que o servidor responde como Apache Tomcat 9.0.93, faixa vulnerável à CVE-2025-24813. Comando de verificação (não intrusivo, apenas inspeção de banner):

```
curl -I https://alvo.example.com/ 2>&1 | grep -i "server\|x-powered-by"
```

Saída esperada: presença do header Server: Apache-Coyote/1.1 ou similar, e correlação posterior com a versão 9.0.93 identificada pelo Nessus. Não foi realizada nenhuma tentativa de exploração efetiva (escrita arbitrária, partial PUT ou desserialização), em respeito ao escopo Blackbox não destrutivo definido na Fase 1. A presença da versão vulnerável foi confirmada por correlação de banner e CPE retornado pelo Nessus.

11.2.2 PoC AZANHA-04 – CSRF Token Estático

Objetivo: confirmar que o valor do csrfToken não é regenerado a cada requisição GET à página de login. Procedimento de verificação (apenas leitura, sem submissão de formulário):

```
# Requisição 1
curl -s https://alvo.example.com/ | grep -oP
'csrfToken"[^>]*value="[^"]+"' > req1.txt

# Requisição 2 (após 60 segundos)
sleep 60
curl -s https://alvo.example.com/ | grep -oP
'csrfToken"[^>]*value="[^"]+"' > req2.txt

# Comparação dos tokens obtidos
diff req1.txt req2.txt
```

Resultado obtido: o token retornado nas duas requisições é idêntico, confirmando que não há regeneração por requisição ou sessão. A confirmação dispensou qualquer tentativa de exploração ativa do CSRF, atendendo ao princípio não destrutivo.

11.2.3 PoC AZANHA-05 – ViewState JSF Exposto

Objetivo: confirmar que o parâmetro javax.faces.ViewState é exposto no HTML e apresenta valor estável entre requisições, indicando possível ausência de proteção HMAC server-side. Procedimento (apenas leitura):

```
curl -s https://alvo.example.com/ | \
  grep -oP 'javax.faces.ViewState[^>]*value="[^"]+"'
```

Resultado obtido: o mesmo valor de ViewState está presente no HTML em dois formulários distintos. A natureza estática do valor é indício de vulnerabilidade, mas a exploração efetiva (ViewState Deserialization) não foi tentada, pois envolveria a construção de payloads serializados maliciosos, o que extrapola o escopo definido.

11.2.4 PoC AZANHA-07 – JWT Validado em Client-Side

Objetivo: confirmar que a lógica de validação do cookie de sessão ocorre no JavaScript do navegador, podendo ser manipulada via DevTools. Procedimento (análise estática do código retornado, sem modificação de cookies):

```
curl -s https://alvo.example.com/ | grep -A 3 'getCookie("SESSION")'
```

Resultado obtido: o trecho de código JavaScript foi identificado conforme descrito na seção 10.3.4, confirmando que a verificação da presença do token JWT ocorre no client-side. A confirmação não envolveu tentativa de bypass efetivo, mas evidencia a fragilidade da camada de verificação.

11.2.5 PoC AZANHA-10 a 13 – Headers de Segurança Ausentes

Objetivo: confirmar a ausência dos headers HTTP de segurança identificados pelo Nikto. Procedimento (inspeção de headers HTTP):

```
curl -I https://alvo.example.com/ 2>&1 | grep -iE \
  "strict-transport-security|content-security-policy|x-content-type-
options|\
   x-frame-options|referrer-policy|permissions-policy"
```

Resultado obtido: nenhum dos cabeçalhos listados retorna na resposta HTTP, confirmando a ausência de hardening básico no servidor. Esta PoC é totalmente passiva (apenas leitura de cabeçalhos via HEAD) e não causa qualquer impacto.

12 PLANO DE MITIGAÇÃO E ROADMAP (FASE 4)

A partir dos achados consolidados na Tabela 10, foi elaborado um plano de mitigação dividido em três horizontes temporais: imediato (hotfix), temporário (workaround) e definitivo, conforme orientação da metodologia adotada e dos frameworks de referência (NIST SP 800-115, OWASP).

12.1 Correções Imediatas (Hotfix)

Aplicáveis a vulnerabilidades de severidade Crítica e Alta com possibilidade de correção rápida, sem impacto sobre a arquitetura da aplicação:

- **AZANHA-01 / 02 / 03:** atualizar o Apache Tomcat para a versão 9.0.99 ou superior, mitigando integralmente todas as 16 CVEs acumuladas — esta única ação resolve a maior parte dos achados críticos;

- **AZANHA-04:** implementar regeneração do CSRF token a cada requisição ou sessão no servidor, eliminando o caráter estático;
- **AZANHA-08:** adicionar as flags HttpOnly e Secure a todos os cookies de sessão (cinco cookies próprios da aplicação);
- **AZANHA-10 a 13:** configurar no servidor web os cabeçalhos HTTP de segurança Strict-Transport-Security, Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Referrer-Policy e Permissions-Policy.

12.2 Correções Temporárias (Workaround)

Aplicáveis quando a correção definitiva demanda planejamento ou janela de manutenção:

- **AZANHA-06:** restringir o targetOrigin do postMessage a uma lista controlada de domínios conhecidos, antes da remoção definitiva do recurso se este não for necessário;
- **AZANHA-09:** enquanto o PrimeFaces não for atualizado para versão recente (>=10.x), aplicar Content Security Policy estrita para bloquear EL injection e desabilitar o componente FileUpload caso não esteja em uso;
- **AZANHA-14:** como medida paliativa, ocultar a URL de ambiente do código HTML público até a refatoração do redirecionamento.

12.3 Correções Definitivas

Aplicáveis a vulnerabilidades que envolvem refatoração de arquitetura ou substituição de componentes:

- **AZANHA-05:** habilitar criptografia/HMAC do ViewState do JSF (parâmetro org.apache.myfaces.SECRET ou equivalente Mojarra) e, idealmente, migrar para tecnologia mais moderna em médio prazo;
- **AZANHA-07:** refatorar a lógica de autenticação para validação exclusivamente server-side, removendo a verificação client-side do cookie JWT;
- **AZANHA-09:** atualizar o PrimeFaces para a versão estável mais recente disponível, com plano de regressão de telas;
- **AZANHA-15:** implementar Subresource Integrity (SRI) em todos os recursos externos carregados via CDN.

12.4 Roadmap de Implementação

A Tabela 11 consolida o roadmap de implementação, com responsáveis sugeridos e prazos estimados.

Tabela 11 – Plano de mitigação e roadmap

ID	Ação	Tipo	Responsável	Prazo
AZANHA-01/02/03	Atualizar Apache Tomcat para 9.0.99+	Hotfix	Infraestrutura	Até 7 dias
AZANHA-04	Regenerar CSRF token por requisição	Hotfix	Desenvolvimento	15 dias
AZANHA-08	Adicionar HttpOnly/Secure aos cookies	Hotfix	Desenvolvimento	Até 7 dias
AZANHA-10 a 13	Configurar headers HTTP de segurança	Hotfix	Infraestrutura	Até 7 dias
AZANHA-06	Restringir targetOrigin do postMessage	Workaround	Desenvolvimento	15 dias
AZANHA-14	Remover URL hardcoded do HTML público	Workaround	Desenvolvimento	15 dias
AZANHA-05	Habilitar criptografia/HMAC do ViewState	Definitiva	Desenvolvimento	45 dias
AZANHA-07	Migrar validação JWT para server-side	Definitiva	Desenvolvimento	45 dias
AZANHA-09	Atualizar PrimeFaces para versão recente	Definitiva	Desenvolvimento	90 dias
AZANHA-15	Implementar Subresource Integrity (SRI)	Definitiva	Desenvolvimento	45 dias

Fonte: Elaborada pelo autor (2025).

12.5 Plano de Retest e Critérios de Aceitação

Após a aplicação das correções, recomenda-se a execução de retestes pontuais (por vulnerabilidade corrigida) e de um teste de regressão integral ao final do roadmap. Os critérios de aceitação para encerramento de cada ID são apresentados na Tabela 12.

Tabela 12 – Critérios de aceitação para retest

ID	Critério de aceitação
AZANHA-01/02/03	Banner do servidor reporta Apache Tomcat 9.0.99 ou superior; Nessus não retorna mais CVEs do grupo Tomcat
AZANHA-04	Tokens CSRF coletados em duas requisições consecutivas apresentam valores distintos

ID	Critério de aceitação
AZANHA-05	Tentativa de modificação do ViewState gera erro de validação (HMAC inválido)
AZANHA-06	targetOrigin do postMessage não contém curinga; mensagem não é entregue a domínios externos
AZANHA-07	Manipulação do cookie de sessão via DevTools não altera o comportamento de autenticação
AZANHA-08	Cookies retornados pelo servidor contêm explicitamente HttpOnly e Secure
AZANHA-09	Versão do PrimeFaces servida pela aplicação é estável e atual
AZANHA-10 a 13	Cabeçalhos HTTP de segurança presentes em todas as respostas; Nikto não os reporta mais como ausentes
AZANHA-14	Inspeção do HTML público não revela URLs internas de ambiente
AZANHA-15	Recursos externos carregados possuem atributo integrity com hash SHA-384

Fonte: Elaborada pelo autor (2025).

13 RESULTADO ESPERADO E ALCANÇADO

Ao final da execução das quatro fases, foi alcançado o objetivo central do Challenge: a entrega de um plano de ação estruturado, ético e tecnicamente fundamentado, sustentado por evidências reproduzíveis e classificação formal de risco. Foram identificadas 16 vulnerabilidades distintas, classificadas conforme CVSS v3.1, com indicação clara de criticidade, vetor de exploração e plano de mitigação.

Como resultados concretos da execução, destacam-se: (i) o mapeamento completo da superfície de ataque externa do ambiente alvo.example.com; (ii) a identificação de uma vulnerabilidade crítica de severidade 9.8 (CVE-2025-24813 no Apache Tomcat 9.0.93); (iii) o levantamento de seis cabeçalhos HTTP de segurança ausentes; (iv) a documentação de nove fragilidades de aplicação web identificadas em análise manual; e (v) a proposição de um roadmap dividido em três horizontes temporais com responsáveis e critérios de aceitação.

Adicionalmente, o trabalho consolidou competências técnicas (uso de ferramentas de pentest, classificação CVSS, escrita de PoC segura) e gerenciais (planejamento, priorização de risco, comunicação técnica) fundamentais à atuação profissional em segurança ofensiva.

14 CONCLUSÃO

A execução do Challenge de Mentoria I permitiu a aplicação prática, em ambiente controlado, dos principais conceitos de Ethical Hacking e Pentest, abrangendo desde o planejamento e a definição de escopo até a validação final dos achados e a proposição de plano de mitigação. O percurso pelas quatro fases consolidou o ciclo completo de um teste de invasão profissional.

Sob o aspecto técnico, foram identificadas vulnerabilidades significativas no ambiente de homologação alvo.example.com, com destaque para a versão desatualizada do Apache Tomcat (9.0.93), a presença de token CSRF estático, a exposição do ViewState do JSF e a ausência de cabeçalhos HTTP de segurança. As 16 vulnerabilidades documentadas foram classificadas formalmente segundo o CVSS v3.1 e acompanhadas de Provas de Conceito seguras, sem qualquer impacto operacional ao ambiente.

Sob o aspecto ético, o trabalho preservou integralmente os limites estabelecidos no escopo, evitando ações destrutivas, tentativas de força bruta ou manipulação de dados. Todas as evidências foram coletadas por meio de técnicas passivas ou ativas controladas, com registro de comandos, parâmetros e horários, garantindo a auditabilidade do processo.

Sob o aspecto pedagógico, o desafio contribuiu para o amadurecimento técnico e profissional, consolidando competências em ferramentas de segurança ofensiva, classificação de risco e escrita técnica em padrão ABNT. Recomenda-se, como continuidade, a realização periódica de pentests sobre o ambiente, idealmente em modalidade Graybox ou Whitebox, para ampliar a profundidade da avaliação e cobrir vetores que não são acessíveis externamente.

Por fim, registra-se o agradecimento à equipe responsável pelo ambiente avaliado pela oportunidade de aplicar, em cenário realista, os conhecimentos adquiridos ao longo do curso.

GLOSSÁRIO

ABNT: Associação Brasileira de Normas Técnicas, entidade responsável pela normalização técnica no Brasil.

Blackbox: modalidade de pentest em que o avaliador não possui qualquer conhecimento prévio sobre o ambiente-alvo, simulando um atacante externo.

CSRF (Cross-Site Request Forgery): técnica de ataque na qual um usuário autenticado é induzido a executar ações não desejadas em uma aplicação web.

CVE (Common Vulnerabilities and Exposures): sistema de identificação padronizada de vulnerabilidades publicamente conhecidas, mantido pela MITRE Corporation.

CVSS (Common Vulnerability Scoring System): sistema aberto para classificação da severidade de vulnerabilidades de segurança, mantido pelo FIRST.

DAST (Dynamic Application Security Testing): testes dinâmicos de segurança realizados sobre a aplicação em execução.

Ethical Hacking: prática autorizada de simulação de ataques para identificar vulnerabilidades de segurança.

Hardening: processo de fortalecimento de um sistema por meio da redução de sua superfície de ataque e aplicação de boas práticas de configuração.

Homologação: ambiente que replica condições de produção, utilizado para testes e validação antes da entrada em operação.

Hotfix: correção imediata aplicada para resolver uma vulnerabilidade crítica.

HSTS (HTTP Strict Transport Security): cabeçalho HTTP que força o uso de HTTPS na comunicação com o domínio.

JWT (JSON Web Token): padrão aberto para criação de tokens de acesso baseados em JSON.

LGPD: Lei nº 13.709/2018 — Lei Geral de Proteção de Dados Pessoais.

MITRE ATT&CK: base de conhecimento global que cataloga táticas, técnicas e procedimentos utilizados por adversários cibernéticos.

NIST: National Institute of Standards and Technology, agência norte-americana referência em padrões de segurança.

OSINT: Open Source Intelligence — coleta de informações a partir de fontes públicas e legalmente acessíveis.

OWASP: Open Worldwide Application Security Project, fundação dedicada à melhoria da segurança de software.

Pentest: abreviação de Penetration Testing; teste de invasão autorizado.

PoC (Proof of Concept): prova de conceito que demonstra a exploração ou existência controlada de uma vulnerabilidade.

RCE (Remote Code Execution): execução remota de código arbitrário no servidor, considerada o impacto máximo em uma vulnerabilidade.

SRI (Subresource Integrity): mecanismo de verificação de integridade de recursos externos carregados por uma página web.

TTPs: Tactics, Techniques and Procedures — táticas, técnicas e procedimentos empregados por adversários.

ViewState: parâmetro utilizado por aplicações JSF (JavaServer Faces) para manter estado entre requisições.

WAF (Web Application Firewall): firewall especializado em proteção de aplicações web contra ataques comuns.

Workaround: solução paliativa temporária aplicada enquanto a correção definitiva é planejada.

XSS (Cross-Site Scripting): vulnerabilidade que permite injeção e execução de scripts maliciosos no contexto do navegador do usuário.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: informação e documentação — trabalhos acadêmicos — apresentação. Rio de Janeiro: ABNT, 2011.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 6023: informação e documentação — referências — elaboração. Rio de Janeiro: ABNT, 2018.

BRASIL. Lei nº 12.737, de 30 de novembro de 2012. Dispõe sobre a tipificação criminal de delitos informáticos. Diário Oficial da União, Brasília, DF, 3 dez. 2012.

BRASIL. Lei nº 12.965, de 23 de abril de 2014. Estabelece princípios, garantias, direitos e deveres para o uso da Internet no Brasil. Diário Oficial da União, Brasília, DF, 24 abr. 2014.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Diário Oficial da União, Brasília, DF, 15 ago. 2018.

FIRST.org. Common Vulnerability Scoring System v3.1: Specification Document. Forum of Incident Response and Security Teams, 2019. Disponível em: https://www.first.org/cvss/v3-1/. Acesso em: 22 nov. 2025.

LOCKHEED MARTIN. The Cyber Kill Chain. Disponível em: https://www.lockheedmartin.com/en-us/capabilities/cyber/cyber-kill-chain.html. Acesso em: 22 nov. 2025.

MITRE CORPORATION. MITRE ATT&CK Framework. 2024. Disponível em: https://attack.mitre.org. Acesso em: 22 nov. 2025.

NATIONAL VULNERABILITY DATABASE. CVE-2025-24813: Apache Tomcat Path Equivalence and Remote Code Execution. NIST NVD, 2025. Disponível em: https://nvd.nist.gov/vuln/detail/CVE-2025-24813. Acesso em: 22 nov. 2025.

NIST. SP 800-115: Technical Guide to Information Security Testing and Assessment. Gaithersburg: National Institute of Standards and Technology, 2008.

OWASP FOUNDATION. OWASP Top 10: The Ten Most Critical Web Application Security Risks. 2021. Disponível em: https://owasp.org/Top10/. Acesso em: 22 nov. 2025.

OWASP FOUNDATION. Web Security Testing Guide (WSTG) v4.2. 2020. Disponível em: https://owasp.org/www-project-web-security-testing-guide/. Acesso em: 22 nov. 2025.

PENETRATION TESTING EXECUTION STANDARD. PTES Technical Guidelines. Disponível em: http://www.pentest-standard.org/. Acesso em: 22 nov. 2025.

TENABLE. Nessus Vulnerability Scanner – User Guide. Disponível em: https://docs.tenable.com/nessus/. Acesso em: 22 nov. 2025.

WEIDMAN, Georgia. Penetration Testing: A Hands-On Introduction to Hacking. San Francisco: No Starch Press, 2014.